# Actividades del prestador y tipos del administrador — explicación del código

Rama `joaco`, sin commitear. Probado de punta a punta con una base de prueba; 15 tests nuevos (32 en total), todos en verde.

**Qué cubre del enunciado:** el prestador publica sus actividades con sus condiciones, organiza fechas y cupos, y consulta las reservas que recibe. El administrador mantiene los tipos de actividad y de equipamiento (alta, consulta, modificación y eliminación).

---

## 1. Recorrido

```
PRESTADOR (rol EMPRESA o PARTICULAR)
/panel/prestador ─► "Mis actividades" ─► /panel/prestador/actividades
                          ├─ "+ Nueva actividad" ─► formulario ─► se crea ─►
pantalla de horarios
                          ├─ "Editar"            ─► mismo formulario (bloqueado si
hay reservas)
                          ├─ "Horarios"          ─► agregar / eliminar fechas
                          └─ "Dar de baja"       ─► desaparece del portal
                  "Reservas" ─► /panel/prestador/reservas (quién reservó y los
datos del grupo)

ADMINISTRADOR
/panel/admin ─► pestaña "Tipos" ─► /panel/admin/tipos (agregar, renombrar,
eliminar)
```

---

## 2. Archivos

### Nuevos

| Capa | Archivo | Para qué |
|---|---|---|
| Service | `services/ActividadService.java` | Reglas de actividades, horarios y reservas recibidas |
| Service | `services/TiposService.java` | Alta, modificación y baja de tipos de actividad y equipamiento |
| Controller | `controllers/PrestadorActividadesWebController.java` | Pantallas del prestador |
| Controller | `controllers/AdminTiposWebController.java` | Pantalla de tipos del admin |

| Capa | Archivo | Para qué |
|---|---|---|
| Repository | `repository/TipoActividadRepository.java` | Acceso a la tabla de tipos de actividad |
| Repository | `repository/TipoEquipamientoRepository.java` | Acceso a la tabla de tipos de equipamiento |
| Config | `config/TiposInicializador.java` | Carga los tipos iniciales al arrancar |
| Vista | `prestador-actividades.html`, `actividad-form.html`, `actividad-horarios.html`, `prestador-reservas.html`, `admin-tipos.html` | Las pantallas |
| Test | `ActividadServiceTest.java`, `TiposServiceTest.java` | Tests unitarios |

### Modificados

| Archivo | Qué cambió |
|---|---|
| `repository/ActividadRepository.java` | Actividades por emprendimiento y "¿alguna actividad usa este tipo?" |
| `repository/ReservaRepository.java` | "¿La actividad / el horario tiene reservas?" y reservas de un emprendimiento |
| `panel-prestador.html` | Las tarjetas "Mis actividades" y "Reservas" ahora funcionan |
| `panel-admin.html` | Menú con la pestaña "Tipos" |

No se tocaron las entidades: `Actividad`, `Horario`, `TipoActividad` y `TipoEquipamiento` ya existían.

---

## 3. Repositorios

**`ActividadRepository`**

```
        List<Actividad> findByEmprendimientoIdOrderByIdDesc(Long emprendimientoId);
        boolean existsByTipoActividad_Id(Long tipoActividadId);
        boolean existsByTiposEquipamiento_Id(Long tipoEquipamientoId);
```

- Spring arma la consulta a partir del **nombre del método**.
- El `_` indica que se entra a la lista `tipoActividad` y se mira el `id` de cada tipo. Así se pregunta: "¿hay alguna actividad que tenga este tipo entre sus categorías?".

**`TipoActividadRepository`** y **`TipoEquipamientoRepository`**

- `findAllByOrderByNombreAsc()` devuelve la lista ordenada alfabéticamente.
- `existsByNombreIgnoreCase(nombre)` impide que existan "Kayak" y "kayak" a la vez.

### `ReservaRepository` (lo agregado)

Se usa `@Query` (JPQL) porque hay que atravesar varias relaciones:

```
        @Query("SELECT COUNT(r) > 0 FROM Reserva r WHERE r.horario.actividad.id =
:actividadId")
        boolean actividadTieneReservas(Long actividadId);
```

- `r.horario.actividad.id` recorre Reserva → Horario → Actividad.
- `horarioTieneReservas(...)` hace lo mismo para un horario concreto (actividad + fecha + hora).
- `reservasDelEmprendimiento(...)` devuelve las reservas de **todas** las actividades del emprendimiento, ordenadas por fecha.

---

## 4. `ActividadService`

El mismo estilo que el resto del proyecto: `@Autowired` y métodos que devuelven el mensaje de error, o `null` si salió bien.

### Constantes

- `DEPARTAMENTOS`: los 19 departamentos. El prestador elige de esta lista, así el texto queda siempre igual y el filtro "Departamento" del buscador funciona.
- `ACTIVA` / `BAJA`: los valores del campo `estado` de `Actividad`.

### Métodos de apoyo

| Método | Qué hace |
|---|---|
| `emprendimientoDe(prestadorId)` | Busca al prestador logueado y devuelve su emprendimiento |
| `puedePublicar(emprendimiento)` | `true` solo si está **ACEPTADO** (RNE #8) |
| `actividadDelPrestador(prestadorId, actividadId)` | Devuelve la actividad **solo si es de ese prestador**; si es de otro, devuelve `null`. Así nadie puede editar actividades ajenas cambiando el número en la URL |
| `tieneReservas(actividad)` | Usa la consulta del repositorio |

### Validación — `validarDatos(...)`

Revisa, en orden: 1. Que título, descripción y departamento no estén vacíos. 2. Que el título tenga como máximo 120 caracteres. 3. Que el departamento sea uno de la lista. 4. Que el precio sea 0 o más. 5. Que la duración sea mayor a 0. 6. Que los minutos previos para reservar sean 0 o más.

### Alta — `crearActividad(...)` (CU-P06)

1. Si el emprendimiento no está aceptado, devuelve `null` y no crea nada.
2. Valida los datos.

3. Crea la `Actividad` con `estado = ACTIVA`, asociada al emprendimiento.
4. `copiarDatos(...)` carga los campos y las **categorías y el equipamiento** elegidos (relaciones n:n del MER) con `findAllById(ids)`.
5. Guarda.

### Edición — `editarActividad(...)` (CU-P07)

- Si la actividad no es suya → "La actividad no existe."
- Si está dada de baja → error.
- **Si ya tiene reservas → no se puede editar (RNE #15).** El motivo: el turista reservó y pagó con un precio y unas condiciones, y no pueden cambiar después.
- Si pasa todo eso, valida, copia los datos y guarda.

### Baja — `darDeBaja(...)` (CU-P08)

- **Baja lógica:** no se borra, se pone `estado = BAJA` (RNE #9).
- Así se conserva el historial de reservas.
- La actividad deja de aparecer en el portal, porque `CatalogoService.estaPublicada` ya excluye las que están en `BAJA`.

### Horarios — `validarHorario(...)` y `agregarHorario(...)` (CU-P09)

`validarHorario` controla: - que la actividad exista y no esté dada de baja; - que la fecha y la hora sean **futuras**; - que la hora de fin sea **después** de la de inicio; - que haya al menos 1 cupo; - que no exista ya un horario con la misma fecha y hora de inicio (es la clave del horario); - si "admite cancelación", que estén los minutos de anticipación y el % de reembolso (entre 0 y 100), como pidió el cliente en el acta (P8–P9).

`agregarHorario` crea el `Horario` con **cupos disponibles = cupos totales**. Las reservas los van descontando.

### Eliminar horario — `eliminarHorario(...)`

- Solo si **nadie lo reservó** (RNE #6).
- Si tiene reservas: "Ese horario ya tiene reservas: no se puede eliminar."

### Reservas recibidas — `reservasRecibidas(prestadorId)` (CU-P11)

Busca el emprendimiento del prestador y llama a `reservasDelEmprendimiento`.

---

## 5. `TiposService` (CU-A05)

| Método | Regla |
|---|---|
| `crearTipoActividad(nombre)` | No vacío, máximo 60 caracteres, sin repetir (sin importar mayúsculas) |
| `renombrarTipoActividad(id, nombre)` | Igual que crear; permite dejar el mismo nombre |
| `eliminarTipoActividad(id)` | **No borra si alguna actividad lo usa** |
| (los mismos tres para equipamiento) | |

**Por qué no se borra un tipo en uso:** es un dato global, compartido por actividades de muchos emprendimientos. Si se borrara, a esas actividades les desaparecería la categoría (ver las decisiones del MER).

**`TiposInicializador`**

- Es un `CommandLineRunner`: se ejecuta solo al arrancar la app, igual que `AdminInicializador`.
- Si las tablas están vacías, carga los tipos de actividad del enunciado (turismo de naturaleza, trekking, aventura, gastronomía, actividad acuática, recorrido histórico, enoturismo, cabalgata, senderismo) y de equipamiento (casco, chaleco salvavidas, bicicleta, equipo de buceo, calzado deportivo, agua).
- Si ya hay tipos, no hace nada.

---

## 6. Controllers

**`PrestadorActividadesWebController` — `@RequestMapping("/panel/prestador")`**

Al colgar de `/panel/prestador`, el **`ControlDeAccesoInterceptor` ya lo protege**: solo entran EMPRESA o PARTICULAR. El id del prestador sale de la sesión (`usuarioId`), nunca del formulario.

| Ruta | Qué hace |
|---|---|
| `GET /actividades` | Lista sus actividades; calcula cuáles tienen reservas para no mostrarles “Editar” |
| `GET /actividades/nueva` | Formulario vacío (si el emprendimiento no está aceptado, vuelve con un aviso) |
| `POST /actividades/nueva` | Crea; si hay error vuelve al formulario **con lo escrito**; si sale bien va a “Horarios” |
| `GET /actividades/{id}/editar` | Formulario con los datos cargados y las categorías marcadas |
| `POST /actividades/{id}/editar` | Guarda los cambios o muestra el error |
| `POST /actividades/{id}/baja` | Baja lógica |
| `GET /actividades/{id}/horarios` | Lista de horarios + formulario para agregar |
| `POST /actividades/{id}/horarios` | Agrega un horario |
| `POST /actividades/{id}/horarios/eliminar` | Elimina un horario (si no tiene reservas) |
| `GET /reservas` | Reservas recibidas |

Conceptos que se usan: - **`@RequestParam(required = false)`:** si falta un campo no da error 400; lo maneja la validación con un mensaje claro. - **`List<Long> tipos`:** cada casilla de categoría se llama `tipos`, así Spring junta las marcadas en una lista. - **`RedirectAttributes.addFlashAttribute`:** el mensaje (“Actividad creada”, etc.) sobrevive a un `redirect:` y se muestra una sola vez. - **Patrón POST → redirect:** después de guardar se redirige, así si el usuario aprieta F5 no se crea la actividad dos veces. -

**`armarActividad(...)`:** arma una `Actividad` que no se guarda, solo para volver a llenar el formulario cuando hay un error.

**`AdminTiposWebController` — `@RequestMapping("/panel/admin/tipos")`**

Cuelga de `/panel/admin`, así que el interceptor solo deja pasar al ADMINISTRADOR.

| Ruta | Qué hace |
|---|---|
| `GET /panel/admin/tipos` | Muestra las dos listas |
| `POST /actividad` · `/actividad/{id}/editar` · `/actividad/{id}/eliminar` | Crear, renombrar o eliminar un tipo de actividad |
| `POST /equipamiento` · `/equipamiento/{id}/editar` · `/equipamiento/{id}/eliminar` | Lo mismo para equipamiento |

Todas terminan en `resultado(...)`, que pone el mensaje de éxito o de error y vuelve a la lista.

---

## 7. Pantallas

| Pantalla | Detalles |
|---|---|
| `prestador-actividades.html` | Tarjeta por actividad con su estado (ACTIVA / DADA DE BAJA), precio y duración; botones Horarios, Editar (si no tiene reservas), Ver en el portal y Dar de baja (con confirmación). Si el emprendimiento no está aceptado, muestra un aviso y oculta “+ Nueva actividad” |
| `actividad-form.html` | El mismo formulario para crear y editar (`th:if="${edicion}"` cambia el título, el botón y la URL). Las categorías y el equipamiento son casillas en forma de “chip”; las marcadas se pintan de naranja con CSS `:has(input:checked)` |
| `actividad-horarios.html` | Lista con “X de Y lugares libres” y la política de cancelación; botón Eliminar solo si nadie reservó. El formulario muestra los campos de cancelación solo al marcar “Admite cancelación” (con JavaScript) |
| `prestador-reservas.html` | Por reserva: actividad, fecha, cantidad de personas, monto, estado, datos del turista, una tabla con los participantes y la observación |
| `admin-tipos.html` | Dos columnas (actividad y equipamiento), cada una con “Agregar” arriba y cada tipo con su campo para renombrar y su botón Eliminar |

Todas usan `umnatural.css` y el mismo menú de su rol.

---

## 8. Tests

Usan JUnit 5 y Mockito: los repositorios se reemplazan por objetos falsos, así no se necesita la base de datos.

`ActividadServiceTest` **(10)** 1. No se crea una actividad con el emprendimiento **pendiente**. 2. Con el emprendimiento **aceptado**, se crea en estado ACTIVA y asociada a él. 3. Validación de los datos obligatorios (título vacío, departamento inventado, precio negativo). 4. **No se edita una actividad con reservas.** 5. No se edita una actividad de **otro** prestador. 6. La baja es **lógica**: cambia el estado y nunca llama a `delete`. 7. Al crear un horario, **cupos disponibles = cupos totales**. 8. Se rechazan los horarios en el pasado o con la hora de fin antes que la de inicio. 9. No se elimina un horario con reservas. 10. Las reservas recibidas son las de su emprendimiento.

`TiposServiceTest` **(5)** 1. No se borra un tipo en uso. 2. Sí se borra un tipo sin uso. 3. No se repiten nombres (aunque tengan espacios o cambien las mayúsculas). 4. Se crea un tipo nuevo. 5. No se borra un equipamiento en uso.

En IntelliJ: clic derecho en `src/test/java` → **Run 'All Tests'**. `contextLoads` necesita tener configurado `DB_PASSWORD`.

---

## 9. Reglas del MER y casos de uso cubiertos

| Regla / CU | Dónde |
|---|---|
| RNE #6 — horario con reservas no se toca | `eliminarHorario` |
| RNE #8 — solo emprendimientos aceptados publican | `puedePublicar`, `crearActividad`, pantallas |
| RNE #9 — baja lógica | `darDeBaja` |
| RNE #15 — actividad con reservas no se edita | `editarActividad` |
| Política de cancelación por horario (acta P8–P9) | `validarHorario`, `agregarHorario` |
| CU-P06, P07, P08, P09, P11 | `ActividadService` + controller |
| CU-A05 — datos maestros | `TiposService` + `AdminTiposWebController` |

## 10. Lo que no se hizo y por qué

- **Cancelar un horario que ya tiene reservas (CU-P10):** implica devolver plata según el % de reembolso, y eso depende del módulo de pagos.
- **Departamento como tabla administrable:** se usó la lista fija de 19. Es más simple y garantiza que el filtro funcione; se puede justificar en la defensa.
- **Pago, comentarios y API pública:** son de otros bloques.

## 11. Preguntas probables en la defensa

- **¿Cómo evitan que un prestador edite la actividad de otro?** `actividadDelPrestador` compara el emprendimiento de la actividad con el del usuario de la sesión; si no coinciden, devuelve `null`.

- **¿Por qué no se puede editar una actividad con reservas?** Para que el turista no termine con un precio o condiciones distintas de lo que aceptó (RNE #15).
- **¿Por qué la baja no borra?** Para conservar el historial de reservas y pagos (RNE #9).
- **¿Quién controla el acceso a estas páginas?** `ControlDeAccesoInterceptor`, por el prefijo de la URL (`/panel/prestador`, `/panel/admin`) y el rol guardado en la sesión.
- **¿Qué pasa si borran un tipo que se usa?** No se puede: `existsByTipoActividad_Id` lo detecta antes de borrar.